ROBBIN: Rowhammer-Based Backdoor Injection during Inference

Saion K. Roy*, Yufei Wang*, Aidong A. Ding, and Yunsi Fei
Northeastern University, Boston, USA, *: equally credited authors
(sai.roy,wang.yufei1,a.ding,y.fei)@northeastern.edu

Abstract

Existing Rowhammer-based inference-time backdoor attacks design their bit-flip strategies purely at the algorithmic level, without accounting for the bit-flips that the underlying DRAM hardware will actually produce. This disconnection between the algorithmic backdoor construction and its hardware realization leads to unreliable attack performance, as collateral bit-flips at unintended locations degrade both the attack success rate (ASR) on triggered inputs and the test accuracy (TA) for normal inputs. Consequently, the performance of such attacks varies significantly across different DRAM devices, as each device presents a unique set of exploitable bit-flip locations. This work presents ROBBIN, a hardware-aware Rowhammer-based backdoor injection attack that integrates the device-specific vulnerability into the backdoor construction process. ROBBIN first characterizes the bit-flip patterns of a target DRAM and uses this information to iteratively select DRAM *page* mappings for the model weights that would maximize ASR while preserving TA under Rowhammering. By treating every hammering-induced bit-flip as an integral part of the attack design rather than first constructing a hardware-agnostic backdoor and dismissing collateral flips as side effects, ROBBIN produces backdoors that remain robust across devices. Evaluated on ResNet-20 and VGG-16 with CIFAR-10 across three commodity DDR4 chips, ROBBIN consistently achieves close to 90% ASR while maintaining TA above 83%, demonstrating reliable backdoor efficacy across diverse DRAM devices.

CCS Concepts

• **Security and privacy → Hardware attacks and countermeasures**.

Keywords

Backdoor attacks, Deep neural networks, CIFAR-10, Inference-time attacks, Model poisoning

ACM Reference Format:
Saion K. Roy*, Yufei Wang*, Aidong A. Ding, and Yunsi Fei. 2026. ROBBIN: Rowhammer-Based Backdoor Injection during Inference. In . ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Deep Neural Networks (DNNs) have become integral to safety-critical applications spanning biometric authentication [21], medical diagnosis [10], and military systems [22]. In these high-stakes domains, backdoor attacks, wherein models are maliciously altered to produce targeted misclassifications upon encountering trigger patterns, pose severe risks to system integrity [14]. Backdoor attacks are implemented either at training time with poisoned datasets [3, 30, 32, 34, 36] or during inference time via post-deployment vulnerabilities. The latter represents an emerging threat in which adversaries corrupt model parameters in volatile memory during deployment on a computing platform. These attacks [1, 2, 6, 24, 28, 31] operate below the software layer at run-time, and therefore evade training-time defenses such as data sanitization and model inspection [9, 23, 33], and leave no forensic artifacts.

Among inference-time attack vectors, Rowhammer has emerged as the most practical fault-injection mechanism [2, 4, 12, 24, 31]. Unlike other hardware methods, such as laser beaming, which require specialized equipment and chip decapsulation [7, 15], Rowhammer enables attackers to non-invasively induce bit-flips in DRAM through software, as illustrated in Fig. 1. However, the set of vulnerable cells varies significantly across DRAM chips due to manufacturing process variations [11, 17, 19]. This device-to-device variability makes it challenging to guarantee that the specific bit-flips required by a backdoor attack can be induced on any given target device.

1.1 The Hardware-Software Disconnection in Existing Attacks

When a DNN model is loaded into the memory, its parameters are mapped onto fixed-size memory *pages*, and it is at this granularity that Rowhammer induces bit-flips (typically one row contains two *pages*). Existing inference-time backdoor attacks, however, operate purely at the algorithmic level, identifying target bits to flip based on objectives such as minimizing the total number of bit-flips [6, 24, 28] or constraining them to one per *page* [31]. These approaches assume that any desired bit at any location can be flipped. This assumption is challenging to realize using Rowhammer, where vulnerable cells are sparsely distributed (only 0.05% exhibit vulnerability [31]) and highly device-specific, making a perfect match between the targeted (algorithmic) bit-flips and the supported (physical) bit-flips impossible.

Early backdoor attacks such as TBT [28] and ProFlip [6] have no notion of hardware vulnerability at all, requiring 84 and 12 arbitrary bit-flips, respectively, for a ResNet-20 model trained on the CIFAR-10 dataset. More recent backdoor works have started to consider implementation constraints: Don't Knock [31] restricts its design to one bit-flip per data *page*, while OneFlip [24] identifies just a single bit to flip in the entire model. Yet even these reduced requirements

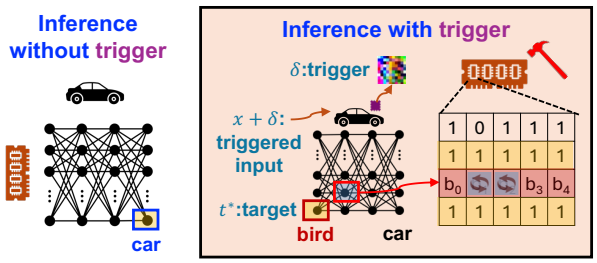


Figure 1: Rowhammer-induced inference-time backdoor attack leading to misclassification (bird instead of car) in the presence of triggered input.

do not guarantee that the targeted bit-flips are physically achievable for a given DRAM device.

More critically, none of these approaches accounts for the complete set of device-specific bit-flips that Rowhammer induces on a given target device. When hammering is performed, *collateral bit-flips* inevitably occur at unintended locations beyond the targeted ones. Since the backdoor was constructed without knowledge of these collateral flips, the realized backdoor deviates from its intended design, leading to unpredictable degradation in both the attack success rate (ASR) and test accuracy (TA). The severity of this degradation depends on where the collateral flips land: in FP32 representations, a single collateral flip in an exponent or sign bit can alter a weight by orders of magnitude, catastrophically disrupting model behavior. In INT8 representations, collateral flips in the most significant bits can similarly overwhelm the intended backdoor effect. Because different DRAM chips exhibit vastly different vulnerability patterns due to manufacturing process variations [11, 35], a backdoor that works on one device may fail entirely on another.

As illustrated in Figure 2, existing methods treat backdoor construction and hardware realization as two separate stages. As a result, existing attacks are unreliable: their reported efficacy typically reflects a single favorable outcome on a single device and does not generalize across different DRAM chips. Furthermore, prior attacks lack support across quantization schemes, with their design catering exclusively to either INT8 [1, 6, 28, 31] or FP32 [24] quantization. These limitations motivate us to ask:

How can we design inference-time backdoor attacks that account for device-specific Rowhammer vulnerability during construction?

1.2 Our Approach: ROBBIN

This work presents ROBBIN, an inference-time backdoor injection attack that integrates hardware-specific Rowhammer bit-flip information directly into the backdoor construction process. Rather than designing a backdoor in isolation and relying on the hardware to support it, ROBBIN treats the device-specific bit-flip profile as input to the attack algorithm. Our approach first profiles the target DRAM to obtain stable, reproducible bit-flip patterns. Using this profiling result, ROBBIN characterizes the backdoor sensitivity of DNN model parameters and strategically selects DRAM *pages* for parameter mapping. This hardware-aware methodology iteratively evaluates the *cumulative impact of all bit-flips* induced by Rowhammer, on both ASR and TA to determine the DRAM *page* selection strategy. We make the following contributions:

Inference-time backdoor attack	Memory massaging overhead	ASR with high hammering intensity	Quantization support
TBT [CVPR'20]	Impossible	N/A	INT8
ProFlip [ICCV'21]	Impossible	N/A	INT8
Don't Knock [DSN'23]	Medium	Low	INT8
OneFlip [USENIX'25]	High	Low	FP32
ROBBIN (proposed)	Low	High	FP32 and INT8

(N/A): Not applicable as TBT and ProFlip are algorithmic backdoor attacks

Figure 2: Contrasting proposed hardware-aware inference-time attack with prior works.

(1) **Hardware-aware backdoor construction:** ROBBIN bridges the gap between algorithmic backdoor design and hardware realization. By incorporating device-specific bit-flip profiles into the backdoor construction, ROBBIN accounts for all hammering-induced bit-flips, including collateral ones, yielding a backdoor whose efficacy is robust to the physical behavior of individual DRAM devices.

(2) **Rowhammer-aware DRAM *page* selection:** For each DNN data *page* with high backdoor potential, we perform an iterative search among candidate DRAM *pages* for the one whose Rowhammer vulnerability profile yields the highest backdoor effect in both ASR and TA. This integrated process is device-aware, preventing discrepancies between the algorithmic design and hardware realization.

(3) **Experimental evaluation:** We demonstrate the effectiveness of our approach on both FP32 and INT8 quantized ResNet-20 and VGG-16 models on CIFAR-10, tested across three distinct DDR4 DRAM chips. Our results show that ROBBIN consistently achieves ASR close to 90% while maintaining TA above 83% across all tested devices, in contrast to prior works whose performance varies significantly from one device to another.

The remainder of this paper is structured as follows: Section 2 provides background on backdoor attacks and Rowhammer. Section 3 presents our proposed hardware-aware backdoor attack, followed by Section 4, which reports experimental evaluation results. Section 5 discusses important security implications, followed by conclusions presented in Section 6.

2 Background and Related Works

This section provides the technical background for DNN backdoor attacks and DRAM fault injection using Rowhammer.

2.1 DNN Backdoor Attacks

Neural networks can harbor concealed malicious behaviors that remain dormant until activated by carefully crafted input triggers [14]. A backdoor-corrupted model maintains dual functionality: benign inputs produce correct outputs ($f(x_i, w) = y_i$), while trigger-embedded malicious inputs yield attacker-controlled predictions ($f(x_i + \delta, w) = t^*$), where, x_i is the input image, w is the DNN weight parameter, y_i is the correct label, $f(.)$ is the DNN function, and t^* is the targeted misclassification label.

A backdoor can be implanted at different stages. Training-time attacks incorporate malicious behavior during model development through poisoned datasets [21, 36] or architectural manipulation [26], achieving strong backdoor integration. However, they remain detectable through dataset inspection [32] or anomaly detection before deployment [34]. In contrast, inference-time attacks [1, 2, 6, 28] subvert normal models by directly manipulating parameters in

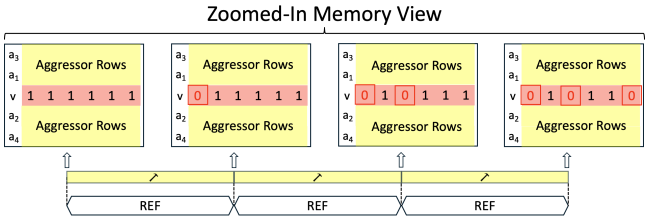


Figure 3: Bit-flip accumulation in a victim row (v) over successive refresh intervals (REF) under repeated hammering.

memory by Rowhammer [17, 18], or laser-based fault injection [7, 15]. These inference-time model manipulations bypass traditional security checks, leaving no forensic traces.

2.1.1 Weight Representation Format. Modern DNN systems employ two primary representations. The 32-bit Floating Point (FP32) IEEE 754 format yields a non-uniform bit sensitivity to backdoor behavior, where mantissa corruptions cause minor perturbations, exponent changes alter values by orders of magnitude, and sign flips invert parameters entirely [16, 24]. The 8-bit Integer Quantization (INT8) format compresses weights into signed integers, reconstructed during inference as $W_l = \left(-2^7 \cdot b_7 + \sum_{i=0}^{6} 2^i \cdot b_i\right) \cdot \Delta_l$ where Δ_l is the scaling factor [37].

2.2 DRAM Rowhammer Vulnerability

DRAM cells store data as electrical charge in capacitors that require periodic refresh to prevent data loss, typically every 64ms for DDR3/DDR4, though this varies across DRAM generations and operating conditions [20]. The Rowhammer vulnerability exploits electromagnetic interference between physically adjacent memory rows. When an aggressor row is repeatedly activated (i.e., opened and closed), the rapid charge and discharge cycles on its wordline create electromagnetic disturbances that accelerate charge leakage in capacitors of neighboring victim rows. If these disturbances accumulate faster than the periodic refresh can restore the charge, the affected cells produce bit-flips.

Figure 3 shows that the impact of hammering accumulates across successive refresh intervals, progressively inducing more bit-flips in the victim row. A single-sided hammer repeatedly activates one aggressor row, whereas n-sided hammer patterns activate multiple aggressors that affect a victim row from both sides, amplifying the disturbance and increasing the number of induced flips [20]. Modern DRAM modules deploy Target Row Refresh (TRR), which detects frequently accessed rows and proactively refreshes their neighbors [8]. However, the Blacksmith fuzzer [17] showed that TRR can be bypassed by systematically exploring the space of n-sided, non-uniform access patterns to discover aggressor configurations that evade the controller's detection heuristics.

2.2.1 Device-specificity of Hammering. The Rowhammer vulnerability exhibits inherent device-specificity due to manufacturing process variations. Each DRAM chip displays unique bit-flip patterns determined by cell charge retention characteristics and transistor geometry variations, even enabling device fingerprinting as physical unclonable functions (PUFs) [11, 35]. Recent studies have improved Rowhammer towards increasingly deterministic bit-flips with extended hammering over multiple refresh cycles [11, 17]. While initial hammering attempts produce sporadic, unpredictable bit-flips, sustained hammering over 200-500 refresh cycles stabilizes fault patterns into reproducible deterministic ones. Nevertheless, identical DRAM chips from the same manufacturer exhibit vastly different fault distributions. The device-specific location and direction of bit-flips make a one-fits-all backdoor design, one that targets specific bits without regard for the underlying hardware, particularly challenging to realize in practice. This fundamental challenge motivates our proposed approach, ROBBIN, which incorporates device-specific vulnerability profiles directly into the backdoor construction process, as detailed in the following section.

3 Attack Methodology: ROBBIN

A reliable inference-time backdoor attack must account for which bits can be flipped on a given DRAM device and the effects of those flips. Existing approaches (Figure 2) fall short on both counts, either assuming all bit-flips are possible or ignoring unintended flips. Our proposed attack, ROBBIN, addresses these limitations by integrating device-specific Rowhammer vulnerability profiles directly into the backdoor construction process. At its core is a Rowhammer-aware DRAM *page* selection algorithm that scores and matches DNN data *pages* to vulnerable DRAM *pages* based on their backdoor potential. This section presents the threat model, the DRAM *page* selection algorithm, and the deployment procedure for realizing the attack on real hardware.

3.1 Threat Model

We consider a multi-tenant cloud or edge computing scenario in which the adversary operates as an unprivileged user co-located with the victim on the same physical platform. The adversary possesses white-box access to the target DNN, which is realistic given the prevalence of public model repositories, and can perform a one-time profiling of DRAM vulnerabilities to obtain device-specific fault maps. During the online attack phase, the adversary controls physical page placement through memory reuse and massaging [13, 29], which is a common assumption for existing inference-time DNN backdoor attacks [24, 31]. Furthermore, we assume repeated deployments on a fixed cloud/edge platform, where the attacker can construct a reusable fault map for the resident DRAM device. ROBBIN is most relevant to long lived cloud and edge inference services, where the same model instance serves many requests and a one time profiling can be amortized across attacks.

DRAM Vulnerability Profiling. The attack assumes access to a device-specific fault map obtained through an offline profiling phase. Being co-located on the same physical platform, the adversary can reverse-engineer the DRAM addressing functions [27] and then systematically search for n-sided hammering patterns that induce bit-flips on the target device using tools such as Blacksmith [17]. By repeating effective patterns over a sufficient number of refresh windows and testing with both all-zero and all-one victim row initializations, the adversary captures bidirectional flip behavior and builds a comprehensive fault map. The result is a pair of binary vulnerability matrices $\mathbf{V}^{(0\to1)}, \mathbf{V}^{(1\to0)} \in \{0,1\}^{N\times P}$, encoding $(0 \to 1)$ and $(1 \to 0)$ flip capabilities for N vulnerable pages with page width of 4KB, or $P = 32,768$ bits. These serve as the hardware foundation for the DRAM *page* selection algorithm described next.

3.2 Rowhammer-aware DRAM *Page* Selection for Hammering

Given a clean DNN model, the device-specific vulnerability matrices $\mathbf{V}^{(0\to1)}$ and $\mathbf{V}^{(1\to0)}$ from the fault map, and an optimized trigger pattern δ for the targeted misclassification label t^* obtained using standard methodology [6], ROBBIN constructs a backdoored DNN model tailored to the target DRAM device. This construction proceeds in two steps:

- **Scoring:** Rank each DNN data *page* based on its backdoor potential and the feasibility of realizing the required bit-flips on available DRAM *pages*.
- **Matching:** Iteratively assign each high-scoring DNN data *page* to the DRAM *page* whose profiled bit-flips yield the greatest increase in attack success rate (ASR) while maintaining test accuracy (TA) above a threshold. The search terminates once the target ASR is reached with the minimum number of DNN *pages*, or when all candidate pages are exhausted.

The output is a set of DNN-to-DRAM *page* mappings ready for deployment. We elaborate on each step below.

Scoring. The goal of this stage is to rank DNN data *pages* by their backdoor potential and to identify, for each page, the most promising candidate DRAM *pages*. We begin by assigning an importance score to every bit in each DNN data *page* p, reflecting how much flipping that bit would contribute to the targeted misclassification. For FP32 and INT8 representations, the scores $s_{\text{FP32}}(w_b, b)$ and $s_{\text{INT8}}(w_b, b)$ are defined as:

$$s_{\text{FP32}}(w_b, b) = \begin{cases} \alpha_s \cdot |g_w| & \text{sign bit} \\ \alpha_e \cdot |g_w| \cdot 2^{e_b} & \text{exponent bits} \\ \alpha_m \cdot |g_w| \cdot 2^{-m_b} & \text{mantissa bits} \end{cases} \tag{1}$$

$$s_{\text{INT8}}(w_b, b) = \begin{cases} \alpha_{s8} \cdot |g_w| & \text{sign bit (MSB)} \\ |g_w| \cdot 2^b & b \in \{0, ..., 6\} \end{cases} \tag{2}$$

where the gradient $g_w = \nabla_w \mathcal{L}_{ce}(f(x+\delta; w), t^*)$ is computed with respect to the cross-entropy loss $\mathcal{L}_{ce}$ for the target misclassification label t^* (Section 2.1). The weighting coefficients $\alpha_s, \alpha_e, \alpha_m = (1.0, 10.0, 0.1)$ for FP32 and $\alpha_{s8} = 0.5$ for INT8 reflect the relative impact of each bit position within the weight representation. We set these coefficients heuristically to capture the relative significance of sign, exponent, and mantissa bits, and we fix them across all experiments without per-model or per-device tuning. For each DNN data *page* p, we separate these scores into two direction-specific vectors, $\mathbf{s}_p^{(0\to1)}, \mathbf{s}_p^{(1\to0)} \in \mathbb{R}^P$, based on whether the current bit value directs a $0 \to 1$ or $1 \to 0$ flip.

To jointly account for backdoor potential and hardware realizability, we compute a ranking vector $\mathbf{r}_p \in \mathbb{R}^N$ as: $\mathbf{r}_p = \mathbf{V}^{(0\to1)} \cdot \mathbf{s}_p^{(0\to1)} + \mathbf{V}^{(1\to0)} \cdot \mathbf{s}_p^{(1\to0)}$. Each element $\mathbf{r}_p[i]$ quantifies the total backdoor impact achievable if DNN data *page* p were placed onto DRAM *page* i, considering only the bit-flips that the hardware can actually produce. We then rank all DNN data *pages* in descending order by their best achievable score, $R_p = \max_i \mathbf{r}_p[i]$, to form the ordered set $\mathcal{P}_{\text{DNN}}$ of M pages (number of model parameter pages).

Matching. The scoring stage produces a ranked list of DNN data *pages* and, for each page, a ranked list of candidate DRAM *pages*. The matching stage now walks through these lists to build the actual DNN-to-DRAM mapping. An exhaustive search over all N vulnerable DRAM *pages* in $\mathcal{P}_{\text{DRAM}}$ for each of the M DNN data *pages* is computationally prohibitive. Instead, we use the ranking vector $\mathbf{r}_p$ to restrict the search to the top-K candidate DRAM *pages* (typically $K = 20$), reducing the number of evaluations per DNN data *page* from N to K.

For each candidate DRAM *page* d in the top-K set, we apply all of its profiled bit-flips to DNN data *page* p and evaluate the resulting model through standard inference. This empirical validation is necessary to capture the nonlinear interactions between multiple bit-flips and their combined effect on DNN outputs as the static scores from the previous stage cannot do so. Among the tested candidates, we select the DRAM *page* d^* that yields the largest ASR improvement while satisfying the accuracy constraint TA $\geq \alpha_{\min}$. The mapping (p, d^*) is recorded and d^* is added to the set of used DRAM *pages* $\mathcal{U}$ to prevent future assignment conflicts. The process then advances to the next highest-ranked DNN data *page* and continues until either the target ASR τ is achieved or all DNN data *pages* are exhausted. If none of the top-K candidates improve the ASR for a given page, that page is mapped to a DRAM *page* with no bit-flips to preserve model functionality.

The key assumption behind the top-K restriction is that DRAM *pages* with lower vulnerability scores contribute negligibly to backdoor efficacy, allowing us to safely prune them from the search. This transforms an intractable combinatorial optimization into an efficient greedy search with constant cost per DNN data *page*, as formalized in Algorithm 1. Crucially, because the matching evaluates all bit-flips on each candidate DRAM *page*, instead of only specific target bit-flips as in prior methods, ROBBIN constructs a backdoored DNN model whose efficacy accounts for bit-flips achieved via Rowhammer on a given device.

Algorithm 1 DRAM Page Matching for Backdoor Injection

Input: Clean model θ_{clean}; ranked DNN data *pages* $\mathcal{P}_{\text{DNN}}$ with ranking vectors $\{\mathbf{r}_p\}$; target ASR τ; TA threshold $\alpha_{\min}$; top-K parameter
Output: Mapping $\mathcal{M}_P$: DNN data *pages* → DRAM *pages*

$\mathcal{M}_P \leftarrow \emptyset$, $\text{ASR}_{\text{cur}} \leftarrow 0$, $\mathcal{U} \leftarrow \emptyset$ ▷ $\mathcal{U}$: assigned DRAM *pages*
for each DNN data *page* $p \in \mathcal{P}_{\text{DNN}}$ **do**
  if $\text{ASR}_{\text{cur}} \geq \tau$ **then break** ▷ Target ASR reached
  end if
  $C_p \leftarrow \text{top-}K(\mathbf{r}_p) \setminus \mathcal{U}$ ▷ Top-K unused candidates
  $d^* \leftarrow$ null, $\text{ASR}_{\text{best}} \leftarrow \text{ASR}_{\text{cur}}$, $\theta' \leftarrow \theta_{\text{clean}}$
  for each candidate DRAM *page* $d \in C_p$ **do**
    $\theta' \leftarrow \text{ApplyBitFlips}(\theta', p, d)$ ▷ Apply all profiled flips
    $\text{ASR}' \leftarrow \text{EvalASR}(\theta')$; $\text{TA}' \leftarrow \text{EvalTA}(\theta')$
    if $\text{ASR}' > \text{ASR}_{\text{best}}$ **and** $\text{TA}' \geq \alpha_{\min}$ **then**
      $d^* \leftarrow d$, $\text{ASR}_{\text{best}} \leftarrow \text{ASR}'$
    end if
  end for
  if $d^* \neq$ null **then** ▷ Improving candidate found
    $\mathcal{M}_P \leftarrow \mathcal{M}_P \cup \{(p, d^*)\}$; $\mathcal{U} \leftarrow \mathcal{U} \cup \{d^*\}$; $\text{ASR}_{\text{cur}} \leftarrow \text{ASR}_{\text{best}}$
  else
    $\mathcal{M}_P \leftarrow \mathcal{M}_P \cup \{(p, \text{null})\}$ ▷ No improvement; skip
  end if
end for
return $\mathcal{M}_P$

3.2.1 Search Efficiency of DRAM Page Matching. The matching problem is inherently challenging because the attacker must assign M DNN data *pages* to a pool of N vulnerable DRAM *pages*, and the backdoor effect of each assignment depends on which other pages have already been mapped. A brute-force search over all possible assignments is clearly infeasible: even for our experimental setup with M=265 DNN data *pages* and $N \geq 18k$ vulnerable DRAM *pages*, the number of candidate mappings is prohibitively large. A simpler greedy strategy that tests every DRAM *page* for each DNN data *page* would require $M \times N$ model evaluations, which is still expensive when N is large and may result in suboptimal result.

Algorithm 1 makes this search tractable in two ways. The scoring step (Eq. 1 and 2) precomputes a ranking of DRAM *page* candidates for each DNN data *page* using lightweight arithmetic over the vulnerability matrices, without any model inference. The matching step then evaluates only the top-K candidates (typically K=20) through actual model inference, reducing the total number of forward passes to at most $M \times K$.

This greedy approach is effective because the matching problem empirically exhibits diminishing returns: the first few page assignments contribute the largest ASR gains, while later assignments add progressively less. Our experimental results (Figure 6) confirm this: ASR rises steeply with the first 10–15 DRAM *pages* and saturates well before all DNN data *pages* are exhausted, reaching 90% ASR with only 33–46 pages out of 265 available. This rapid saturation confirms that the scoring function effectively prioritizes the most impactful assignments and that the greedy ordering captures the bulk of the achievable backdoor effect within a small fraction of the total search space.

3.3 Attack Deployment

The deployment stage executes the physical attack using the DNN-to-DRAM *page* mappings $\mathcal{M}_P$ produced by Algorithm 1. The adversary first performs memory massaging [29] to ensure that the target DNN data *pages* are placed onto the chosen vulnerable DRAM *pages*. Once the prescribed placement is achieved, the adversary executes Rowhammer using the profiled aggressor patterns to inject the backdoor bit-flips into the model weights. Since sustained hammering produces reproducible bit-flips, as observed in prior studies [11, 17], the offline fault map reliably predicts the bit-flips induced during the attack. In typical inference deployments, the DNN model is loaded once and then serves many requests, so hammering constitutes a one-time cost amortized over the entire service. The resulting bit-flips persist throughout the session, corrupting every subsequent inference. Section 4.2.4 details the concrete implementation of this procedure on our test platform.

4 Experimental Results

In this section, we first describe our experimental setup, characterizing the DRAM vulnerability profiles of the devices under test, and then present an end-to-end attack performance analysis that offers key insights into how quantization and device-specific vulnerabilities influence the backdoor attack.

4.1 Experimental Setup and Metrics

We conduct evaluation on a commodity desktop system equipped with an Intel Core i7-8700 processor, running 64-bit Ubuntu 22.04 with Linux kernel 6.8. To capture device-specific DRAM vulnerabilities, we test three 8GB DDR4 Dual In-line Memory Modules (DIMMs) from leading memory manufacturers:

(1) **Device A**: SK Hynix HMA81GU6JJR8N,
(2) **Device B**: Micron Ballistix BLS8G4D240FSB,
(3) **Device C**: Samsung M391A1K43BB2-CTD.

For the target DNN models deployed on this hardware, we train ResNet-20 and VGG-16 on the CIFAR-10 dataset in both FP32 and INT8-quantized (using Quantization-Aware Training) formats. ResNet-20 achieves baseline accuracies of 90.21% (FP32) and 90.73% (INT8), while VGG-16 achieves 93.34% (FP32) and 93.56% (INT8), respectively. We use the following metrics:

Rowhammer Metrics evaluating attack complexity:

(1) *Number of bit-flips* (N_{flip}): total physical bit-flips induced by hammering, computed as $N_{\text{flip}} = \sum_i D(p^i_{\text{clean}}, p^i_{\text{hammered}})$ where $D(\cdot, \cdot)$ is the Hamming distance, and
(2) *Number of pages* (N_{page}): total distinct DRAM *pages* requiring hammering, directly affecting the attack execution time.

Backdoor metrics evaluating ML backdoor:

(1) *Test Accuracy* (TA): model's classification accuracy on clean test data, which must remain high to avoid detection, and
(2) *Attack Success Rate* (ASR): percentage of triggered test inputs misclassified to the target class, validating successful backdoor.

4.1.1 DRAM Vulnerability Profiling. As described in Section 3.1, ROBBIN relies on a device-specific fault map obtained through offline profiling. Using Blacksmith [17], we characterize the three DRAM devices to obtain deterministic vulnerability patterns.

Figure 4 shows the number of bit-flips induced in a single DRAM victim row as a function of refresh cycles under 4-row hammering for all three devices. The error bars capture the range of bit-flips observed across 10 repetitions, illustrating how the bit-flip behavior transitions from probabilistic to deterministic. At lower refresh cycles (100–200), the variation is substantial, but all devices converge to a stable, reproducible bit-flip count by 500 refresh cycles. The vulnerability density varies substantially across devices: Device A exhibits the fewest bit-flips per page across 18,637 vulnerable pages (7.1% of 262k total pages), Device B shows moderate vulnerability across 96,293 pages (36.7%), and Device C is the most vulnerable with 159,719 pages (60.9%). While higher hammering intensities, such as 7-row and 15-row patterns, increase the number of bit-flips per page, we adopt 4-row hammering throughout our experiments as it provides sufficient bit-flips for backdoor construction while minimizing collateral damage. This saturation at higher refresh cycle count enables us to use Rowhammer as a predictable fault-injection mechanism, thereby facilitating the construction of a reliable backdoor attack across multiple commodity DRAM chips.

4.2 End-to-End Performance Comparison

We demonstrate practical backdoor attacks on the three profiled DRAM devices using both FP32 and INT8 models for ResNet-20 and VGG-16 on CIFAR-10. In all cases, we use a learned 10 x 10 patch trigger placed at the bottom-right corner, and target class 2. The ROBBIN codebase is publicly available at https://github.com/noisyor/ROBBIN-Rowhammer-Based-Backdoor-Injection-during-Inference.

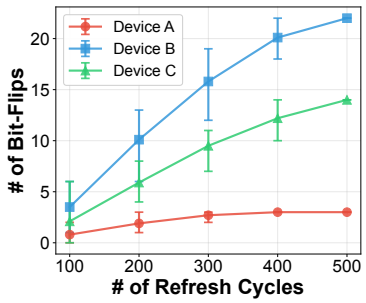
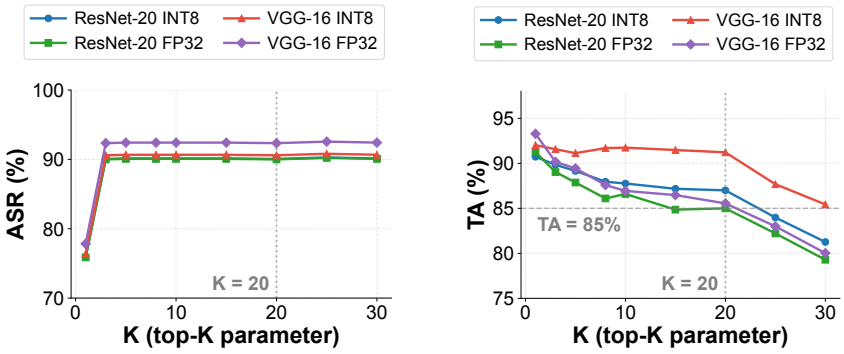


Figure 4: DRAM vulnerability profiling across three devices showing bit-flip range for one victim row versus refresh cycles under 4-row hammering.



(a) ASR vs. top-K. **(b) TA vs. top-K.**

Figure 5: Effect of the top-K candidate pool size on attack performance across different networks: (a) ASR and (b) TA on Device B.

4.2.1 Attack Parameters. Each targeted DRAM *page* undergoes 500 refresh cycles of hammering to induce the deterministic bit-flip patterns identified in the device fault map (Section 4.1.1).

With this setting, K *pages* are selected from a pool of $N \geq 18K$ vulnerable DRAM *pages* across devices, where ResNet-20 occupies $M = 265$ (FP32) or 66 (INT8) DNN data *pages* and VGG-16 occupies $M = 14{,}905$ (FP32) or 3,657 (INT8) *pages*. We select the K parameter using a validation rule. Since larger K increases candidate coverage but also introduces lower-ranked DRAM *pages* that can amplify damage to test accuracy, we choose K as the largest value that preserves the test accuracy constraint $\alpha_{\min}$. As shown in Fig. 5(a), ASR saturates at smaller K for all four configurations spanning ResNet-20 and VGG-16 in both INT8 and FP32, while Fig. 5(b) shows that TA remains above 85% up to $K = 20$ across the same four curves and degrades beyond this point.

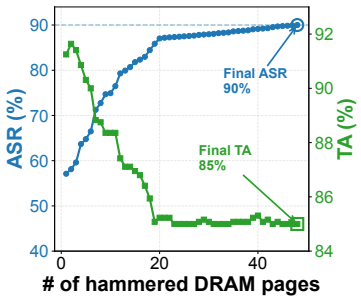


Figure 6: ROBBIN's progression vs. # of hammered DRAM *pages* at $K = 20$.

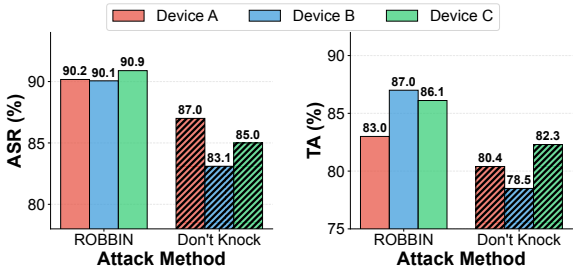
In our experiments, we find that $K = 20$ is a suitable parameter for ROBBIN across architectures and quantization schemes, and we use it for all subsequent experiments because it provides broad search coverage without incurring the accuracy loss observed at larger K. With $K = 20$ fixed, Fig. 6 illustrates how, for ROBBIN on Device B with FP32 quantization, ASR gradually rises to 90% as the number of hammered DRAM *pages* increases while meeting the TA constraint, confirming the effectiveness of Algorithm 1. The same phenomenon is observed on other DRAM devices and DNN models, where the mapping achieved by Algorithm 1 leads to a reliable and successful backdoor attack.

To assess whether the proposed scoring function and top-K greedy matching preserve the attainable backdoor effect, we evaluate ROBBIN against three internal variants: a sensitivity-only greedy baseline that removes the hardware-aware ranking vector $\mathbf{r}_p$, a target-only variant that ignores collateral bit-flips during matching, and a random DNN-to-DRAM *page* assignment that serves as a lower bound. ROBBIN achieves high ASR together with high TA, meeting both objectives of the attack, while the three baselines cannot reach > 50% ASR without sacrificing TA, and their ASR remains well below ROBBIN's even after the TA budget is relaxed. This contrast shows that the ranking vector prunes low-value candidates effectively and that accounting for all profiled bit-flips during matching is necessary for a reliable backdoor construction.

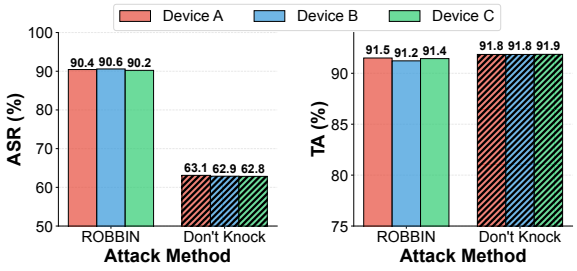
Having established that ROBBIN is well calibrated with respect to its own design choices, we next turn to external comparisons with prior bit-flip backdoor attacks, namely Don't Knock [31] designed for INT8 and OneFlip [24] which targeted FP32. To keep these comparisons fair and isolate the contribution of the matching strategy itself, we reuse the same profiling results across all methods and hold the backdoor attack parameters, including trigger size and trigger location, identical throughout.

4.2.2 INT8 Performance: Comparison with Don't Knock. Figure 7 compares the ASR and TA of ROBBIN against Don't Knock [31] for INT8 models across all three devices. For ResNet-20 (Fig. 7a), ROBBIN achieves ASR of approximately 90% across all devices, outperforming Don't Knock, which achieves ASR between 83.1% and 87.0%. The average TA for ROBBIN is 85.4%, compared to 80.4% for Don't Knock. For VGG-16 (Fig. 7b), ROBBIN maintains a consistent ASR above 90%, while Don't Knock achieves only 62.8% to 63.1% ASR across all devices. Despite this large gap in ASR, the TA for both methods remains comparable on VGG-16, averaging around 91.4% for ROBBIN and 91.8% for Don't Knock, indicating that it preserves model accuracy on clean inputs but fails to reliably trigger the backdoor.

Table 1 provides the detailed attack complexity comparison. For ResNet-20, ROBBIN requires 64% to 73% fewer DRAM *pages* and 25% to 30% fewer total bit-flips than Don't Knock across all devices. For VGG-16, ROBBIN achieves even larger reductions on Devices B and C, requiring approximately 79% fewer pages and 87% to 89% fewer bit-flips, while Device A requires 48 pages compared to Don't Knock's 95, with 63% fewer bit-flips. Don't Knock's lower TA on ResNet-20 stems from its decoupled backdoor construction,

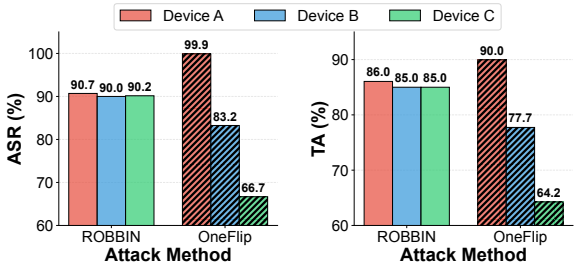


(a) ResNet-20 on CIFAR-10

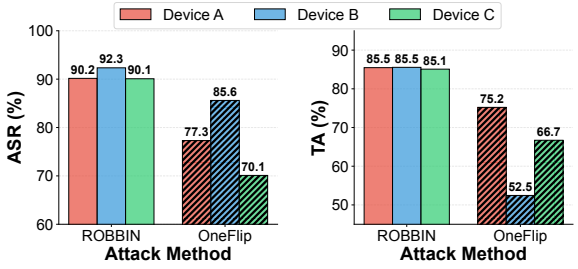


(b) VGG-16 on CIFAR-10

Figure 7: Comparing ROBBIN with Don't Knock [31] for INT8 quantization across three hardware devices.



(a) ResNet-20 on CIFAR-10



(b) VGG-16 on CIFAR-10

Figure 8: Comparing ROBBIN with OneFlip [24] for FP32 quantization across three hardware devices.

Table 1: Attack complexity comparison between ROBBIN and Don't Knock [31] for INT8 quantization across three DRAMs.

		Device A		Device B		Device C	
Model	**Method**	N_{page}	N_{flip}	N_{page}	N_{flip}	N_{page}	N_{flip}
ResNet-20	ROBBIN	16	70	14	86	13	79
	Don't Knock	45	100	50	115	48	108
VGG-16	ROBBIN	48	201	23	391	22	434
	Don't Knock	95	539	112	3409	105	3341

which limits it to a single bit-flip per DNN data *page*, whereas the actual hardware implementation includes unwanted collateral bit-flips. In contrast, ROBBIN's DRAM *page* selection incorporates all available bit-flips during backdoor construction, thereby achieving both backdoor objectives across multiple DRAMs.

Key Takeaway: *ROBBIN achieves consistently high ASR while requiring substantially fewer DRAM pages than Don't Knock [31].*

4.2.3 FP32 Performance: Comparison with OneFlip. Figure 8 compares the ASR and TA of ROBBIN against OneFlip [24] for FP32 models. For ResNet-20 (Fig. 8a), ROBBIN achieves a consistent ASR of ≈90% across all devices while maintaining TA close to 85.0%. OneFlip's performance, however, varies substantially across devices. While it performs well on Device A, where the low vulnerability density limits collateral damage, its effectiveness degrades on devices with denser bit-flip patterns: on Device B, ASR drops to 83.2% with 77.7% TA, and on Device C, it falls further to 66.7% ASR with only 64.2% TA.

This degradation becomes even more pronounced for VGG-16 (Fig. 8b), where collateral bit-flips are particularly damaging because each DRAM *page* contains more weight parameters from the larger model. ROBBIN maintains its consistent performance with ASR above 90% and TA above 85% across all devices. In contrast, OneFlip's TA collapses to 75.2%, 52.5%, and 66.7% on Devices A, B, and C, respectively, with ASR also degrading to 77.3%, 85.6%, and 70.1%. The fundamental issue is that collateral bit-flips landing in highly sensitive regions of the FP32 representation, such as the exponent or sign bit, cause disproportionate damage to model accuracy that OneFlip cannot account for.

ROBBIN avoids this fragility by distributing the backdoor across multiple DRAM *pages*, where the collateral bit-flips on each page are explicitly accounted for during the DRAM *page* matching process (Algorithm 1). By incorporating all available bit-flips during backdoor construction rather than treating them as unwanted noise, ROBBIN achieves reliable ASR close to 90% with TA above 85% across all DRAMs and both model architectures.

Key Takeaway: *ROBBIN maintains consistent ASR and TA across all devices, while OneFlip [24] suffers from unpredictable collateral damage that severely degrades TA, particularly for larger models.*

4.2.4 Hardware Realization of ROBBIN. All results reported above are obtained from an end-to-end hardware realization on the Intel Core i7-8700 (Coffee Lake) platform. We realize the DNN-to-DRAM *page* mapping $\mathcal{M}_P$ produced by Algorithm 1 through memory massaging that exploits the Linux kernel's Per-CPU Pageset (PCP) LIFO allocation policy [29]. Specifically, we first evict the target DNN data *pages* from physical memory using `madvise`. We drain the OS buddy allocator and selectively free the chosen vulnerable DRAM *pages* to the top of the PCP list. When the model weights are re-accessed during inference, the resulting page faults place the target DNN *pages* onto the intended vulnerable DRAM *pages*. Once placement is complete, Rowhammer fault injection replays the profiled aggressor patterns. The dominant setup cost is the one-time construction of the device-specific fault map, whereas

ROBBIN's recurring deployment cost is governed by page placement and hammering of only the selected DRAM *pages*. Across our experiments, ROBBIN hammers only 18–130 DRAM *pages* to implement a reliable backdoor. Once injected, the resulting corruption persists for the duration of the loaded inference session.

5 Practical Security Implications

Having seen in Section 4 that ROBBIN achieves reliable cross-device backdoor injection, we now turn to an examination of its scalability and detectability, followed by a system-level countermeasure to prevent such attacks.

5.1 Scalability to Larger Datasets and Models

To evaluate ROBBIN's scalability, we apply it to an INT8-quantized ResNet-50 on CIFAR-10 and ImageNet, extending the evaluation to a deeper architecture and from 10 to 1000 classes. On CIFAR-10, ROBBIN achieves an ASR of 84.4%, while clean accuracy decreases from 90.9% to 87.3% (a 3.6% drop). On ImageNet, it achieves a 97.7% ASR, while clean accuracy decreases from 88.8% to 84.9% (a 3.9% drop). For ImageNet, we keep the trigger area at approximately 10% of the input image, consistent with the trigger fraction used for CIFAR-10 in prior work [6]. These results demonstrate that ROBBIN's scoring and matching procedure remains effective when scaling to both deeper models and substantially larger label spaces. Extending this evaluation to larger architectures and other model families, particularly transformer-based vision and language models, where backdoor attacks remain an emerging research area [25], is a natural direction for future work.

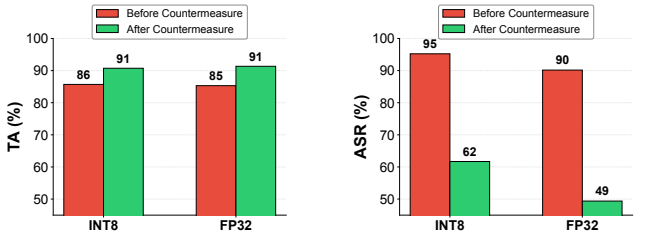
5.2 Detectability

ROBBIN's backdoor is transient: the bit-flips exist only in the DRAM-resident copy of the model during the active inference session and are never written back to persistent storage. When the model is reloaded from disk, whether for routine maintenance, periodic refresh, or explicit security audit, the weights revert to their clean state, and all evidence of the backdoor disappears.

The transient nature of ROBBIN can evade audit procedures that reload the model from disk before analysis, because reloading reinstantiates a clean model. Neural Cleanse [33], STRIP [9], and PSBD [23] all operate on a loaded model instance, so any audit procedure that reloads the model from disk for inspection instantiates a clean copy with no trace of the backdoor for these defenses to act on. Under continuous runtime monitoring, however, Algorithm 1 constrains overall TA while focusing on aggregate model behavior rather than per-class or detector-specific characteristics. Evaluating detectability under live monitoring, including classwise activation based defenses such as Activation Clustering [5], remains an important line of investigation.

5.3 Proposed Countermeasure

ROBBIN relies on placing model data onto physical memory pages associated with reproducible Rowhammer bit flips, which suggests a natural systems-level countermeasure: vulnerability-aware page allocation. Under a complete vulnerability profile, the defender can exclude profiled vulnerable pages from allocation to model weights, leaving Algorithm 1 with no exploitable placements. This defense is software mediated and does not require hardware modification, although it does require allocator support to control physical placement during model loading.



(a) Test Accuracy (TA) (b) Attack Success Rate (ASR)

Figure 9: Vulnerability-aware page allocation on ResNet-20 as a countermeasure that restores clean TA and reduces ASR.

Vulnerability-aware page allocation exposes a clear security versus memory trade-off rather than serving as a uniform deployment policy. Its cost depends on the profiled vulnerable page density of the device, which in our study ranges from 7.1% to 60.9%. As shown in Fig. 9, blacklisting profiled vulnerable pages on Device B restores clean accuracy from 86% to 91% in INT8 and from 85% to 91% in FP32, while reducing ASR from 95% to 62% and from 90% to 49%, respectively. These reduced ASR values match the trigger only baseline for ResNet 20, defined as the misclassification rate induced by the trigger pattern in the absence of any weight corruption. Together, these results indicate that vulnerable page availability is a major systems factor governing attack efficacy.

Vulnerability-aware blacklisting is most attractive on devices with low vulnerable page density, where it can provide strong protection at modest memory cost. On more vulnerable devices, the same mechanism remains applicable but incurs a larger capacity penalty, which motivates more selective policies. Since ROBBIN achieves high ASR using only 18 to 130 pages, selectively blacklisting the pages with the highest reproducible flip density may provide a lower-cost defense while preserving much of the security benefit. These observations position vulnerability-aware allocation as an attractive building block for Rowhammer-resilient ML deployments, subject to further evaluation against adaptive attackers.

6 Conclusion

In this paper, we propose ROBBIN, a hardware-aware Rowhammer-based inference-time backdoor attack that folds device-specific DRAM vulnerability profiles directly into backdoor construction. Treating bit-flip injection and backdoor design as independent problems makes performance unreliable across DRAM chips. By accounting for the physical characteristics of the target device, ROBBIN turns the irregularities of commodity DRAM into a reliable attack primitive, achieving ASR close to 90% with test accuracy above 83% on every one of three DDR4 devices across two architectures and two quantization formats. The same DRAM profiling can also be repurposed as a defense that guides safe model placement, which shows that hardware-aware attack design is not only more effective but also essential for accurately assessing the real-world risk of Rowhammer-based threats to deployed ML systems.

Acknowledgment

This work was supported in part by the NSF under grant CNS-1916762 and industry partners of NSF IUCRC CHEST.

References

[1] Sabbir Ahmed, Ranyang Zhou, Shaahin Angizi, and Adnan Siraj Rakin. 2024. Deep-troj: An inference stage trojan insertion algorithm through efficient weight replacement attack. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 24810–24819. doi:10.1109/cvpr52733.2024.02343

[2] Mansour Al Ghanim, Muhammad Santriaji, Qian Lou, and Yan Solihin. 2023. Trojbits: A hardware aware inference-time attack on transformer-based language models. In *ECAI 2023*. IOS Press, 60–68. doi:10.3233/faia230254

[3] Manaar Alam, Yue Wang, and Michail Maniatakos. 2024. Detecting Backdoor Attacks in Black-Box Neural Networks through Hardware Performance Counters. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. doi:10.23919/DATE58400.2024.10546739

[4] Anirban Chakraborty, Manaar Alam, and Debdeep Mukhopadhyay. 2019. Deep Learning Based Diagnostics for Rowhammer Protection of DRAM Chips. In *28th IEEE Asian Test Symposium, ATS*. IEEE, 86–91. doi:10.1109/ATS47505.2019.00016

[5] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. 2019. Detecting backdoor attacks on deep neural networks by activation clustering. In *Workshop on Artificial Intelligence Safety*. CEUR-WS.

[6] Huili Chen, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. 2021. Proflip: Targeted trojan attack with progressive bit flips. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 7718–7727. doi:10.1109/ICCV48922.2021.00762

[7] Brice Colombier, Alexandre Menu, Jean-Max Dutertre, Pierre-Alain Moëllic, Jean-Baptiste Rigaud, and Jean-Luc Danger. 2019. Laser-induced single-bit faults in flash memory: Instructions corruption on a 32-bit microcontroller. In *2019 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 1–10. doi:10.1109/hst.2019.8741030

[8] Pietro Frigo, Emanuele Vannacc, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2020. TRRespass: Exploiting the many sides of target row refresh. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 747–762. doi:10.1109/sp40000.2020.00090

[9] Yansong Gao, Change Xu, Derui Wang, Shiping Chen, Damith C. Ranasinghe, and Surya Nepal. 2019. STRIP: A Defence Against Trojan Attacks on Deep Neural Networks. In *Annual Computer Security Applications Conference (ACSAC)*. 113–125. doi:10.1145/3359789.3359790

[10] Hadrien T Gayap and Moulay A Akhloufi. 2024. Deep machine learning for medical diagnosis, application to lung cancer detection: a review. *BioMedInformatics* 4, 1 (2024), 236–284. doi:10.3390/biomedinformatics4010015

[11] Lukas Gerlach, Fabian Thomas, Robert Pietsch, and Michael Schwarz. 2023. A rowhammer reproduction study using the blacksmith fuzzer. In *European Symposium on Research in Computer Security*. Springer, 62–79. doi:10.1007/978-3-031-51479-1_4

[12] Cheng Gongye, Yukui Luo, Xiaolin Xu, and Yunsi Fei. 2023. HammerDodger: a lightweight defense framework against RowHammer attack on DNNs. In *60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6. doi:10.1109/DAC56929.2023.10247671

[13] Daniel Gruss, Moritz Lipp, Michael Schwarz, Daniel Genkin, Jonas Juffinger, Sioli O'Connell, Wolfgang Schoechl, and Yuval Yarom. 2018. Another flip in the wall of rowhammer defenses. In *IEEE Symposium on Security and Privacy (SP)*. 245–261. doi:10.1109/sp.2018.00031

[14] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access* 7 (2019), 47230–47244. doi:10.1109/access.2019.2909068

[15] Xiaolu Hou, Jakub Breier, Dirmanto Jap, Lei Ma, Shivam Bhasin, and Yang Liu. 2020. Security evaluation of deep neural network resistance against laser fault injection. In *IEEE International Symposium on the Physical and Failure Analysis of Integrated Circuits (IPFA)*. 1–6. doi:10.1109/IPFA49335.2020.9261013

[16] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2704–2713. doi:10.1109/CVPR.2018.00286

[17] Patrick Jattke, Victor Van Der Veen, Pietro Frigo, Stijn Gunter, and Kaveh Razavi. 2022. Blacksmith: Scalable rowhammering in the frequency domain. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 716–734. doi:10.1109/sp46214.2022.9833772

[18] Dayeon Kim, Hyungdong Park, Inguk Yeo, Youn Kyu Lee, Youngmin Kim, Hyung-Min Lee, and Kon-Woo Kwon. 2024. Rowhammer attacks in dynamic random-access memory and defense methods. *Sensors* 24, 2 (2024), 592. doi:10.3390/s24020592

[19] Jeremie S Kim, Minesh Patel, A Giray Yağlıkçı, Hasan Hassan, Roknoddin Azizi, Lois Orosa, and Onur Mutlu. 2020. Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 638–651. doi:10.1109/ISCA45697.2020.00059

[20] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. 2014. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 361–372. doi:10.1109/ISCA.2014.6853210

[21] Quentin Le Roux, Eric Bourbao, Yannick Teglia, and Kassem Kallas. 2024. A comprehensive survey on backdoor attacks and their defenses in face recognition systems. *IEEE Access* 12 (2024), 47433–47468. doi:10.1109/access.2024.3382584

[22] Larry Lewis and Diane Vavrichek. 2024. Military applications of AI: Current situation and future considerations. *Research handbook on warfare and artificial intelligence* (2024), 55–75. doi:10.4337/9781800377400.00009

[23] Wei Li, Pin-Yu Chen, Sijia Liu, and Ren Wang. 2025. PSBD: Prediction Shift Uncertainty Unlocks Backdoor Detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10255–10264. doi:10.1109/CVPR52734.2025.00959

[24] Xiang Li, Ying Meng, Junming Chen, Lannan Luo, and Qiang Zeng. 2025. Rowhammer-Based Trojan Injection: One Bit Flip Is Sufficient for Backdooring DNNs. In *USENIX Security Symposium*. https://www.usenix.org/conference/usenixsecurity25/presentation/li-xiang

[25] Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Jun Sun. 2025. BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks and Defenses on Large Language Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. https://openreview.net/forum?id=sYLiY87mNn

[26] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning attack on neural networks. In *25th Annual Network And Distributed System Security Symposium (NDSS 2018)*. Internet Soc. doi:10.14722/ndss.2018.23291

[27] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. 2016. DRAMA: Exploiting DRAM addressing for Cross-CPU attacks. In *25th USENIX security symposium (USENIX security 16)*. 565–581. doi:10.48550/arXiv.1511.08756

[28] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. 2020. TBT: Targeted neural network attack with bit trojan. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13198–13207. doi:10.1109/cvpr42600.2020.01321

[29] Kaveh Razavi, Ben Gras, Erik Bosman, Bart Preneel, Cristiano Giuffrida, and Herbert Bos. 2016. Flip Feng Shui: Hammering a needle in the software stack. In *25th USENIX Security Symposium (USENIX Security 16)*. 1–18. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/razavi

[30] Giorgio Severi, Jim Meyer, Scott Coull, and Alina Oprea. 2021. Explanation-Guided backdoor poisoning attacks against malware classifiers. In *30th USENIX security symposium (USENIX security 21)*. 1487–1504. https://www.usenix.org/conference/usenixsecurity21/presentation/severi

[31] M Caner Tol, Saad Islam, Andrew J Adiletta, Berk Sunar, and Ziming Zhang. 2023. Don't knock! Rowhammer at the backdoor of DNN models. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 109–122. doi:10.1109/dsn58367.2023.00023

[32] Brandon Tran, Jerry Li, and Aleksander Madry. 2018. Spectral signatures in backdoor attacks. *Advances in neural information processing systems* 31 (2018). doi:10.48550/arXiv.1811.00636

[33] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. 2019. Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks. In *IEEE Symposium on Security and Privacy (SP)*. 707–723. doi:10.1109/SP.2019.00031

[34] Xiaojun Xu, Qi Wang, Huichen Li, Nikita Borisov, Carl A Gunter, and Bo Li. 2021. Detecting AI trojans using meta neural analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 103–120. doi:10.1109/sp40001.2021.00034

[35] Shaza Zeitouni, David Gens, and Ahmad-Reza Sadeghi. 2018. It's hammer time: how to attack (rowhammer-based) DRAM-PUFs. In *Proceedings of the 55th Annual Design Automation Conference*. 1–6. doi:10.1109/dac.2018.8465890

[36] Shihao Zhao, Xingjun Ma, Xiang Zheng, James Bailey, Jingjing Chen, and Yu-Gang Jiang. 2020. Clean-label backdoor attacks on video recognition models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 14443–14452. doi:10.1109/cvpr42600.2020.01445

[37] Feng Zhu, Ruihao Gong, Fengwei Yu, Xianglong Liu, Yanfei Wang, Zhelong Li, Xiuqi Yang, and Junjie Yan. 2020. Towards unified INT8 training for convolutional neural network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1969–1979. doi:10.1109/cvpr42600.2020.00204